



Procesadores del Lenguaje

Grado en Ingeniería Informática

Entrega final.

Grupo 81/ Equipo 107

Año académico 2025/2026

Miguel Amiel Gil

| 100525454 | 100525454@alumnos.uc3m.es

Chen Xu

| 100522395 | 100522395@alumnos.uc3m.es

Especificaciones FrontEnd	3
Variables globales	3
Función MAIN	3
Impresión de cadenas	3
Impresión de expresiones y cadenas	3
Operadores, precedencia y asociatividad	3
Estructura de control WHILE	3
Estructura de control IF	3
Variables locales	3
Estructura de control FOR	3
Estructura de control Switch/Case	3
Funciones	3
Implementación de vectores	4
Especificaciones Backend	4
Variables globales	4
Función MAIN	4
Impresión de cadenas	4
Impresión de valor entero	4
Asignación de variable	4
Estructura de control IF	4
Correspondencia operadores	4

Especificaciones FrontEnd

Variables globales

Para implementar las variables globales, hemos añadido una nueva producción a nivel de axioma. Concretamente, `axioma ::= INTEGER dec_var ';' .` Primero se busca la palabra clave `INTEGER` y a continuación nos movemos al no terminal `dec_var` donde se va procesando el resto de la declaración. Concretamente, se busca un primer identificador, el cual es obligatorio. A continuación se continúa analizando mediante el no terminal `continue_ID`. `Continue_ID` permite la asignación de la variable o simplemente su inicialización. Finalmente, `continue_comma` permite seguir declarando más variables o finalizar mediante la lectura de `lambda`.

```
C/C++
axioma:
    INTEGER dec_var ';' { printf ("%s\n", $2.code) ; } r_axioma {;}

dec_var:
    IDENTIF continue_ID { if(strcmp(dentro_funcion,"global") == 0){
        sprintf(temp, "(setq %s %s", $1.code,
        $2.code);}else{ sprintf(temp, "(setq %s %s %s",dentro_funcion,$1.code,
        $2.code); añadir_variable_local($1.code);} $$code = gen_code(temp);}

continue_ID:  continue_comma { sprintf(temp, "0)%s", $1.code);
        $$code = gen_code(temp);}

        | '=' NUMBER continue_comma { sprintf(temp, "%d)%s", $2.value,
        $3.code);
        $$code = gen_code(temp); }

        | '=' '-' NUMBER continue_comma { sprintf(temp, "-%d)%s",
        $3.value, $4.code);
        $$code = gen_code(temp); }

        | '=' '+' NUMBER continue_comma { sprintf(temp, "%d)%s",
        $3.value, $4.code);
        $$code = gen_code(temp); }

        | '[' expresion ']' continue_comma { sprintf(temp, "(make-array
        %s)%s", $2.code, $4.code);$$code = gen_code(temp); }
    ;
continue_comma:  ',' dec_var { sprintf(temp, "\n%s", $2.code);
        $$code = gen_code(temp);}

    | { $$code = " ";};
```


Operadores, precedencia y asociatividad

Se fueron añadiendo de forma progresiva, los operadores aritméticos y lógicos. Estos son: AND, OR, NOT, !=, =, <, <=, >, >=, %, +, *, /, -.. Para ello, se modificó el no terminal expression añadiendo los operadores correspondientes. Algunos de estos operadores, fueron añadidos como palabras clave.

C/C++

```
expression:  operand { ; }
            | '(' '-' expression expression ')'      { printf (" - ") ; }
            | '(' AND  expression expression ')' { printf(" %s ", $2.code);}

            | '(' OR   expression expression ')' { printf(" %s ", $2.code);}

            | '(' NOT  expression ')'              { printf(" 0= ");}

            | '(' NOT_EQUAL expression expression ')' { printf(" = 0= ");}

            | '(' '='  expression expression ')'      { printf(" = ");}

            | '(' '<'  expression expression ')' { printf(" < ");}

            | '(' LESS_EQUAL expression expression ')' { printf(" %s ",
$2.code);}
            | '(' '>'  expression expression ')' { printf(" > ");}

            | '(' GREATER_EQUAL expression expression ')' { printf(" %s ",
$2.code);}

            | '(' MOD expression expression ')' { printf(" %s ", $2.code);}
            | '(' '+' expression expression ')' { printf(" + ");}
            | '(' '*' expression expression ')' { printf(" * ");}
            | '(' '/' expression expression ')' { printf(" / ");}
            | '(' '-' expression ')' { printf (" negate ") ; } // Unary minus
operator in Lisp
;
```

Por otro lado, se especificó la precedencia para conseguir que los operadores se ejecuten en el orden deseado. Para ello, se siguió la tabla de precedencia que se encuentra en el siguiente enlace:

https://en.cppreference.com/c/language/operator_precedence

C/C++

```
%right '=' // es la ultima operacion que se debe realizar
%left OR
%left AND
%left EQUAL NOT_EQUAL
%left '<' LOE '>' GOE
%left '+' '-' // menor orden de precedencia
%left '*' '/' '%' // orden de precedencia intermedio
%left UNARY_SIGN '!'
```



```

if_instrucciones:
    sentencia {$$.code = $1.code;}
    | RETURN expresion ';' {sprintf(temp, "(return-from %s
%s)", dentro_funcion, $2.code); $$code = gen_code(temp);}
    ;

if_sentencia:
    if_instrucciones { $$code = $1.code;}
    | multiples_sentencias { sprintf(temp, "(progn %s)", $1.code);
        $$code = gen_code(temp);}
    ;

multiples_sentencias:
    if_instrucciones multiples_sentencias_cont { sprintf(temp,
"%s\n%s", $1.code, $2.code);
        $$code = gen_code(temp);}
    ;

multiples_sentencias_cont:
    if_instrucciones {$$ = $1;}
    | multiples_sentencias { $$ = $1;}
    ;
else_cont:    {$$.code = gen_code("");}
    | ELSE '{' if_sentencia '}' { sprintf(temp, "%s\n", $3.code);
        $$code = gen_code(temp);}
    ;

```


Variables locales

Para las variables locales, hemos creado la tabla de variables locales que almacena cada uno de los nombres de dichas variables. Para ello, hemos usado un array de strings, un contador de variables y una variable dentro_main que permite saber el scope en el que nos encontramos. Esta última variable es necesaria para poder saber si printear main_variable o variable al utilizar una variable dentro de la función main. Considerando estos añadidos, modificamos nuestro código C de la siguiente forma:

```
C/C++
char *local_variables[2048]; // Tabla de variables locales
int local_variables_counter = 0; // Contador de variable locales que
tenemos
int dentro_main = 0; // Variable para saber si nos encontramos dentro
del main o no

int es_local(char *var_name){
    for(int i = 0; i < local_variables_counter; i++){
        if(strcmp(var_name,local_variables[i]) == 0){
            // Si coincide el nombre de la variable con alguna del
            array, entonces es local.
            return 1;
        }
    }
    // Si no hubo ninguna coincidencia, entonces es global
    return 0;
}

void añadir_variable_local(char *nombre){
    // Añadimos la variable al array y aumentamos el contador
    local_variables[local_variables_counter] = gen_code(nombre);
    local_variables_counter++;
}
```

Es importante destacar las funciones es_local y añadir_variable_local que permiten comprobar si una variable es local, es decir, se encuentra en la tabla y también añadir una variable a la tabla correspondiente.

Estructura de control FOR

Después, hemos implementado la estructura de control FOR. Hemos dado por hecho que en la inicialización de la variable, esta será la variable local y no global. Para conservar la lógica anterior de los setf y setq, hemos primero inicializado la variable antes de entrar al loop for y en esta inicialización se comprueba si la variable ya fue declarada antes. Si la variable ya fue declarada, se usa setf y en otro caso sería setq. Hemos separado las expresiones en expresiones de comparación y expresiones aritméticas para especificar en la expresión de la estructura de control. Además hemos añadido a FOR, INC y DEC a los tokens.

Por último, otra idea importante es que, durante el desarrollo del bucle FOR, tuvimos que dividir el no terminal "expresion" en 3 no terminales distintos: expr_condicional, expr_others y término. Esto fue necesario para poder, a través de un único no terminal, referirnos a todas las expresiones condicionales.

Concretamente se modificó el código, generando estas nuevas reglas:

```
sentencia ::= FOR '(' for_var ';' expr_condicional ';' for_operator
for_var   ::= IDENTIF '=' expresion
expr_condicional -> expresiones condicionales de la gramática
original y añadidas es decir, operadores( > >= < <= not and...)
for_operator ::= INC '(' IDENTIF ')' 'while_cont' | DEC '(' IDENTIF
')' ')' 'while_cont'
```

Estructura de control Switch/Case

Para implementar la estructura de control Switch/Case, primero, añadimos dos nuevos tokens SWITCH y CASE para tratar esas palabras clave. Además, añadimos una nueva regla en sentencia para gestionar el SWITCH. Se ha añadido en el no terminal sentencia ya que es este el encargado de generar todas las posibles instrucciones permitidas dentro de una función, entre ellas el switch case.

Concretamente, la regla añadida fue:

```
sentencia ::= SWITCH '(' IDENTIF ')' switch_cont
```

```
switch_cont ::= '{' CASE switch_val ':' r_sentencia BREAK ';' switch_cont2 '}'
```

```
switch_val ::= '+' NUMBER %prec UNARY_SIGN |  
              '-' NUMBER %prec UNARY_SIGN |  
              NUMBER
```

```
switch_cont2:  
    lambda |  
    DEFAULT ':' r_sentencia BREAK ';' |  
    CASE switch_val ':' r_sentencia BREAK';'
```

Observamos que para la implementación del switch, se han requerido de distintos no terminales auxiliares. Concretamente, switch_cont, switch_val, y switch_cont2.

Switch_cont se utiliza para crear el caso inicial del switch, es decir, especificar el primer case. Al estar lidiando con el caso base, asumimos que al menos se va a requerir de un case dentro del mismo.

A continuación, implementamos switch_cont2 el cual contiene la continuación de dicho switch. Este no terminal permite crear más cases de forma recursiva y finalizar el switch ya sea añadiendo un default o simplemente lambda(sin default). Para la correcta implementación del switch, se consideró el no terminal switch_val. Este contiene todos los valores posibles que se pueden utilizar en el case del switch. Estos son números con o sin signo.

Funciones

Para la implementación de las funciones, modificamos la gramática añadiendo nuevos no terminales con sus correspondientes normas para poder declararlas correctamente. Para su elaboración, se ha asumido que estas funciones se declaran antes de la función main. Esto es coherente con la idea de que una función debe estar declarada(definida para nuestro lab) para poder ser usada en las funciones localizadas en partes inferiores.

func_params_declaration contiene la declaración de los parámetros posibles para una función. Estos son, lambda, es decir, que no tenga parámetros o de nuevo, un tipo de dato y la variable correspondiente. De esta manera, se permite añadir parámetros de forma recursiva hasta que se desee.

Como no estábamos seguros de que tipos de datos se permiten como parámetros para las funciones, hemos considerado tres tipos básicos de parámetros: int, float y char. Para ello, hemos definido dos nuevos tokens FLOAT y CHAR y hemos reutilizado el token INTEGER.

Además, se puede observar como en sentencia se ha añadido RETURN expresion ';' para gestionar el posible return en cualquier lugar de la función. Esto se ha combinado con las reglas r_sentencia como r_sentencia_anidada, anteriormente mencionada, para poder considerar también ese caso.

Para realizar llamadas a funciones, se ha añadido una nueva regla en sentencia que utiliza un no terminal llamado func_call que recoge la sintaxis necesaria. De esta manera, podemos llamar a funciones en medio de una función, como si fueran una instrucción más. Sin embargo, también es necesario que se pueda usar una llamada a una función como expresión en ifs, printf's y otras funciones. Por lo tanto, también se añadió una regla en expresion que genera func_call para lograr este fin. En resumen, se tienen las siguientes reglas:

```
expresion ::= func_call  
sentencia ::= func_call
```

```
func_call ::= IDENTIF '(' func_params_call ')'  
func_params_call ::= lambda | expresion func_params_call_cont
```

func_call permite llamar a una función. Actualmente, hay un problema con este código que no sabemos si tiene que ser implementado, se pueden llamar a funciones no declaradas. Esto será preguntado en la próxima sesión. Con el código actual, además, no se controla que el número de parámetros sea el mismo que los de la función que se está llamando. De esta forma, se permite la adición de parámetros de forma recursiva e indeterminada.

En la declaración de la función se hace la impresión del código. Lo movimos de main a la declaración de cada función para reducir la carga de memoria. Se hace una acción semántica al final que imprime la función entera.

Implementación de vectores

Para crear arrays como variables globales o locales, es decir declararlos, se ha añadido a `continue_id` la regla:

```
continue_id ::= '[' expresion ']' continue_comma
```

Con ello, hemos añadido a las posibles declaraciones anteriores también la declaración del array usando expresiones. De la misma manera, se ha añadido a `sentencia` una regla idéntica a la que fue definida anteriormente para la asignación de variables. En este caso, el parser utiliza el token del lookahead para diferenciar entre las dos reglas:

```
sentencia ::= IDENTIF '=' expresion ';'
sentencia ::= IDENTIF '[' NUMBER ']' '=' expresion ';'

```

Esto nos permite diferenciar entre una asignación de una variable y de un array. Tras esto, necesitamos poder reconocer expresiones que utilicen los ítems de dentro de un array. Por ello, se han añadido nuevas reglas que permiten reconocer expresiones en las que se usen ítems del array mediante índices. Esto dio lugar a las siguientes reglas:

```
continue_ID ::= '[' expresion ']' continue_comma
array_index ::= IDENTIF '[' expresion ']'

```

Especificaciones Backend

Para realizar la implementación del backend de la práctica final, usamos la gramática ya proporcionada en el archivo `back0i.y` y la completamos un poco para cumplimentar los puntos del enunciado y reconocer el lenguaje Lisp. Para ello añadimos los siguientes tokens:

- AND
- OR
- NOT
- NOT_EQUAL
- LESS_EQUAL
- GREATER_EQUAL
- MOD

Por ello, también añadimos sus equivalentes a la tabla de palabras claves, junto con `setq` y `setf`. En las reglas de `expression1`, concretamente en las que aparecen los tokens `SETF` y `PRINC`, las completamos usando no terminales. En el caso de `PRINC`, usamos un nuevo no terminal llamado `princ_param` para permitir el uso de `expression` o de `STRING`. Para el no terminal `expression`, añadimos las expresiones faltantes siendo éstas `AND`, `OR`, `NOT`, `NOT_EQUAL`, `'=`, `'<`, `LESS_EQUAL`, `'>`, `GREATER_EQUAL`, `MOD`, `'+`, `'*` y `'/'`.

Una vez reconocido el lenguaje intermedio, podemos proceder a la traducción. La traducción del lenguaje intermedio Lisp al final Gforth es directo, es decir en cada producción se imprime el valor directamente ya que este usa una pila. Por ello, en cada regla, la acción semántica tratará de poner el formato adecuado para la traducción (de infijo a postfijo cambiando el orden) y hará la traducción de los tokens directamente con la equivalencia. Si una regla lleva a un no terminal, este no se imprimirá ya que esto se hará en sus propias producciones.

Variables globales

Hicimos la traducción literal de Lisp a Gforth. Como la gramática para el reconocimiento del lenguaje Lisp ya estaba hecho, solo añadimos la acción semántica:

```
C/C++
>(' SETQ IDENTIF number '){ printf("%d %s !\n", $4.value, $3.code);}
// This is the declaration of a variable which in Forth has to be of
global scope
```

Función MAIN

Hicimos la traducción literal de Lisp a Gforth. Como la gramática para el reconocimiento del lenguaje Lisp ya estaba hecho, solo añadimos la acción semántica:

C/C++

```
'(' MAIN ')' { printf (" main\n") ; }  
| '(' DEFUN MAIN { printf(": main"); } '(' ')' exprSeq ')' '  
{printf(";"); }
```

Impresión de cadenas

Hicimos la traducción literal de Lisp a Gforth. Como la gramática para el reconocimiento del lenguaje Lisp ya estaba hecho, solo añadimos la acción semántica:

C/C++

```
'(' PRINT STRING ')' { printf(" .\" %s\" cr", $3.code); }
```

Impresión de valor entero

Hicimos la traducción literal de Lisp a Gforth. En este caso, tuvimos que completar la regla para el reconocimiento del lenguaje Lisp. Esto lo hicimos con una nueva no terminal llamada princ_param para permitir diferenciar entre un princ de una expresión o de una cadena:

C/C++

```
'(' PRINC princ_param ')' { ; }
```

princ_param:

```
expression { printf(" . ");}  
| STRING { printf(".\" %s\"", $1.code);}  
;
```

Asignación de variable

Hicimos la traducción literal de Lisp a Gforth. Tuvimos que completar la regla para el reconocimiento del lenguaje Lisp y le añadimos la siguiente acción semántica:

C/C++

```
'(' SETF IDENTIF expression ')' { printf(" %s !\n", $3.code); }
```

Estructura de control IF

Hicimos la traducción literal de Lisp a Gforth. Como la gramática para el reconocimiento del lenguaje Lisp ya estaba hecho, solo añadimos la acción semántica:

```
C/C++
 '(' ifHead expression1 ')' { printf (" THEN\n") ; }

| '(' ifHead expression1 { printf (" ELSE\n") ; } expression1 ')' {
printf (" THEN\n") ; }
;

ifHead: IF expression { printf (" IF ") ; }
;
```

Correspondencia operadores

Hicimos la traducción literal de Lisp a Gforth. Como comentamos anteriormente, añadimos todos los tokens necesarios para las expresiones faltantes al igual que las reglas faltantes de las expresiones:

```
C/C++
%token AND
%token OR
%token NOT
%token NOT_EQUAL
%token LESS_EQUAL
%token GREATER_EQUAL
%token MOD

| '(' AND expression expression ')' { printf(" %s ", $2.code);}
| '(' OR expression expression ')' { printf(" %s ", $2.code);}
| '(' NOT expression ')' { printf(" 0= ");}
| '(' NOT_EQUAL expression expression ')' { printf(" = 0= ");}
| '(' '=' expression expression ')' { printf(" = ");}
| '(' '<' expression expression ')' { printf(" < ");}
| '(' LESS_EQUAL expression expression ')' { printf(" %s ", $2.code);}
| '(' '>' expression expression ')' { printf(" > ");}

| '(' GREATER_EQUAL expression expression ')' { printf(" %s ",
$2.code);}
| '(' MOD expression expression ')' { printf(" %s ", $2.code);}
| '(' '+' expression expression ')' { printf(" + ");}
| '(' '*' expression expression ')' { printf(" * ");}
```



```
| '(' '/' expression expression ')' { printf(" / ");}  
| '(' '-' expression ')' { printf (" negate " ; }
```